

Project Title

Legacy Logistics Rescue Mission

Objective

The objective of this project is to optimize and refactor a failing supply chain database system for Legacy Logistics. The goal is to reduce API response times to under 50ms and achieve a benchmark score above 90% without upgrading hardware, by improving database design, queries, and architecture.

Project Scope

This project focuses on database performance optimization in a simulated production environment using PostgreSQL. The work includes query analysis, indexing, normalization, handling unstructured data, partitioning large tables, and implementing caching techniques.

Project Plan

Phase 1: Environment Setup and Baseline Analysis

- Install required tools (Docker, Python, Git, Database Client).
- Clone the project repository and start the Docker-based simulation.
- Allow the database to fully seed with shipment and telemetry data.
- Run the benchmark script to record initial performance results.
- Save baseline benchmark screenshots for documentation.

Phase 2: Performance Analysis

- Connect to the database using a database client.
- Identify slow queries using EXPLAIN ANALYZE.
- Detects performance issues such as sequential scans and high query costs.

Phase 3: Database Optimization

- Create appropriate B-Tree and composite indexes to eliminate sequential scans.
- Normalize redundant data by separating repeated fields into new tables.
- Convert unstructured invoice data from TEXT to JSONB.
- Create GIN indexes on JSONB columns for faster search.
- Partition large telemetry tables by date to reduce query load.
- Implement materialized views for frequently used dashboard queries.

Phase 4: Testing and Verification

- Apply each optimization using migration scripts without database downtime.
- Re-run benchmark tests after each major change.
- Ensure response times and benchmark scores meet project targets.

Phase 5: Documentation and Submission

- Organize all SQL migration scripts .
- Collect before and after benchmark screenshots.
- Write the final technical report explaining problems, solutions, and results.

Expected Outcome

- API response times reduced to below 50ms.
- Benchmark score improved to above 90%.
- Optimized, scalable, and well-structured database system.

Constraints

- No hardware upgrades allowed.
- No database downtime permitted.
- Limited system resources (0.5 CPU, 512MB RAM).